

Reproducing Token Superposition

Eoghan Flanagan
Independent Researcher
eoghanflanagan@hotmail.com

Claude
Anthropic

Abstract

We reproduce the core finding of *Efficient Pre-Training with Token Superposition* [6]. That paper introduces Token-Superposition Training (TST), a two-phase pre-training method. We re-implement TST from scratch using a GPT-2 medium (345 M parameter) architecture trained on FineWeb-Edu, and compare a 20 000-step TST run (superposition bag size $s = 6$, ratio $r = 0.3$) against a 40 000-step standard baseline at equal FLOPs per step. Our TST run reaches a validation cross-entropy of 3.41 nats; the baseline requires 38,250 steps to reach the same loss, corresponding to a $1.91\times$ speedup in gradient steps and in total FLOPs.

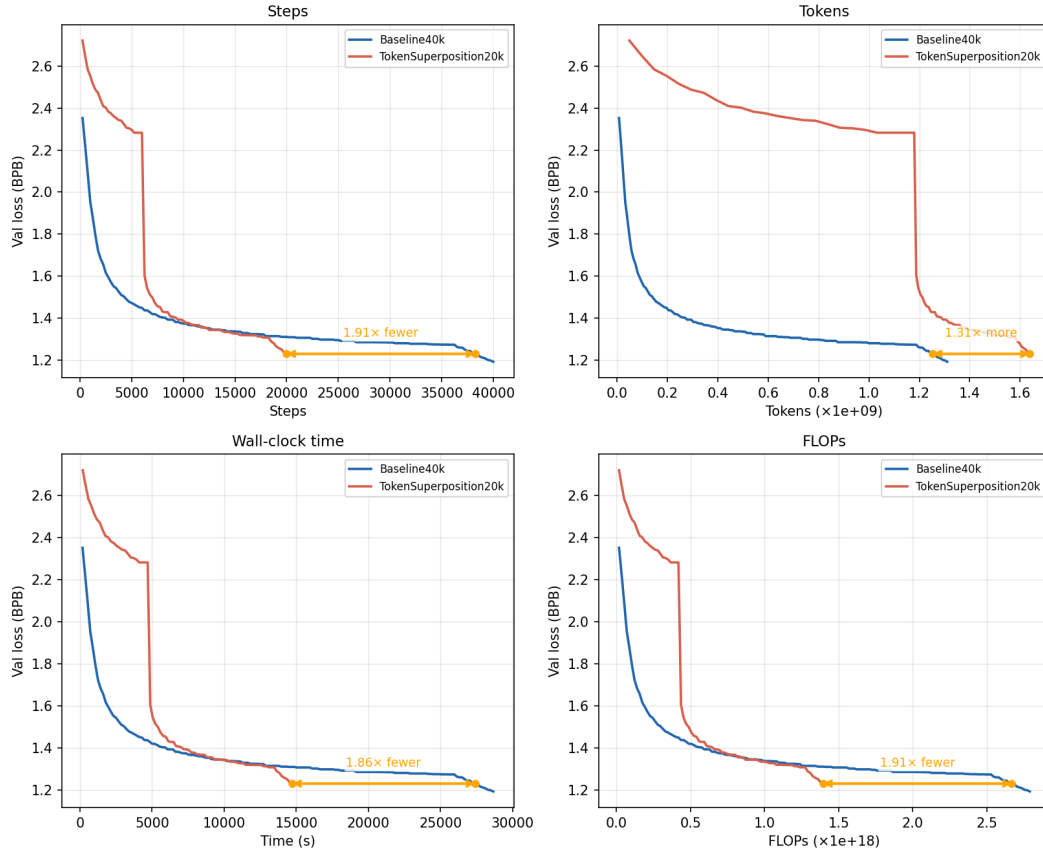


Figure 1: Isoloss curves for validation loss (bits per byte) comparing standard autoregressive pre-training (**Baseline40k**) and Token-Superposition Training (**TokenSuperposition20k**, $s=6$, $r=0.3$). Each curve shows the best validation loss achieved up to that resource level.

1 Introduction

Peng et al. [6] introduce Token-Superposition Training (TST), a drop-in modification to the standard autoregressive pre-training loop that increases data throughput without changing per-step FLOPs, the model architecture, the optimizer, the tokenizer, or the dataset. The method proceeds in two phases:

1. **Superposition phase** (r fraction of total steps): contiguous data tokens are grouped into non-overlapping bags of size s . The embedding of each bag is the mean of its constituent token embeddings, resulting in a sequence that is s times shorter than the raw token sequence. The model is then trained with a multi-hot cross-entropy (MCE) objective that treats each output position as predicting the entire next bag of tokens. Because the transformer operates on the compressed sequence of length L/s rather than L , a data sequence s times longer is fed per step to maintain equal FLOPs relative to baseline.
2. **Recovery phase** ($1 - r$ fraction of total steps): the checkpoint from phase 1 is resumed with standard next-token cross-entropy and sequence length L , with all TST-specific code removed.

The authors report consistent improvements over equal-FLOPs baselines at model scales from 270 M to 10 B parameters.

In this work, we independently re-implement TST and reproduce the method’s central claim: that a TST run reaches a given validation loss strictly faster (in gradient steps and in FLOPs) than a standard autoregressive baseline trained at the same per-step compute budget. We do not reproduce the full experimental sweep from Peng et al. [6]; our goal is a faithful single-setting reproduction sufficient to validate the core claim.

2 Implementation

2.1 Model

We use a GPT-2 medium architecture [7] with 24 transformer layers, residual dimension $d = 1024$, 16 attention heads, and FFN width 4,096. The vocabulary follows GPT-2 (50,257 tokens, BPE tokenizer). We tie the input embedding matrix to the output projection (LM head), giving approximately 345 M parameters. Dropout is disabled throughout.

2.2 Dataset

Training and validation data are taken from FineWeb-Edu [4], pre-tokenized with the GPT-2 tokenizer and stored in the nanoGPT uint16 binary shard format [2]. We use all available training shards and a dedicated validation shard.

2.3 Training Setup

Both runs use the following shared hyperparameters:

- Batch size: 32 sequences; latent sequence length: 1,024 tokens.
- Optimizer: AdamW [3], $\beta_1 = 0.9$, $\beta_2 = 0.95$, weight decay 0.1, fused CUDA kernel.
- Learning rate: 6×10^{-4} with a Warmup-Stable-Decay (WSD) schedule [1]: linear warmup for 200 steps, stable plateau, linear decay to 10% of peak for the last 10% of steps.
- Gradient clipping: global norm 1.0.
- Precision: bfloat16 autocast; FP32 accumulation inside the MCE loss.

Baseline (Baseline40k). Standard next-token prediction for 40,000 steps. Each step processes $32 \times 1,024 = 32,768$ data tokens.

Token Superposition (TokenSuperposition20k). $s = 6$, $r = 0.3$: the first 6,000 steps are the superposition phase (data sequence length 6,144 = $6 \times 1,024$, MCE loss), and the remaining 14,000 steps are the recovery phase (data sequence length 1,024, standard CE loss). The FLOPs per step are identical to the baseline because the transformer operates on the latent sequence of length 1,024 in both phases; only the data throughput differs.

FLOPs accounting. Following standard practice, per-step FLOPs are estimated as $6 \times N \times B \times L$, where $N \approx 345$ M is the parameter count, $B = 32$ the batch size, and $L = 1,024$ the latent sequence length. This gives $\approx 6.98 \times 10^{13}$ FLOPs per step, identical for both runs.

Infrastructure. All runs were executed on a single NVIDIA H100 80 GB SXM5 GPU via Modal [5], with `gradient_checkpointing` enabled for the medium-size model.

2.4 Multi-hot Cross-Entropy Loss

Our MCE loss follows the simplified form from Peng et al. [6]:

$$\mathcal{L}_{\text{MCE}}(\mathbf{z}, \mathbf{y}) = \frac{1}{s} \sum_{y \in \mathbf{y}} \mathcal{L}_{\text{CE}}(\mathbf{z}, y). \quad (1)$$

We compute $\log \sum_i \exp(z_i)$ once per latent position (rather than s times), reducing memory by sharing the log-partition function across the s terms in the bag.

2.5 Input Superposition

In the forward pass, when `bag_size > 1`, the input token sequence of length $B \times (s \cdot L)$ is reshaped to $B \times L \times s$, and the s token embeddings per position are averaged in FP32 before being cast back to bfloat16:

```
1 B, SL = input_ids.shape
2 L = SL // bag_size
3 raw = self.transformer.wte(input_ids.reshape(B * SL)).float()
4 embeds = raw.reshape(B, L, bag_size, -1).mean(dim=2).to(raw.dtype)
```

Listing 1: Input superposition in our implementation

This matches the averaging approach described by Peng et al. [6] and shares embeddings and LM head weights unmodified across both phases, which the authors identify as critical for the recovery to succeed.

3 Results

3.1 Quantitative Summary

Table 1 summarises the final metrics for both runs.

Run	Steps	FLOPs ($\times 10^{18}$)	Wall-clock (hours)	Data tokens (billions)	Val CE (nats)	Val BPB
Baseline40k	40,000	2.79	8.0	1.31	3.31	1.193
TokenSuperposition20k	20,000	1.40	4.1	1.64	3.41	1.231
Baseline (at iso-loss)	38,250	2.67	7.65	1.25	3.41	—

Table 1: Final metrics for both runs. The iso-loss row reports the earliest step at which the baseline first achieved validation CE ≤ 3.41 nats (the TST run’s final validation CE), determined from wandb logs.

The TST run reaches its final validation CE of 3.41 nats at step 20,000. The baseline first reaches this same loss level at step 38,250, giving a speedup of $38,250/20,000 = 1.91\times$ in gradient steps and, since FLOPs per step are equal, an identical speedup in total FLOPs.

The TST run consumed more raw data tokens than the baseline at iso-loss (1.64 B vs. 1.25 B), consistent with the original paper’s observation that TST trades higher data consumption for lower compute cost. The TST run also completed in approximately half the wall-clock time (4.1 h vs. ≈ 7.65 h for the baseline at iso-loss), reflecting the halved step count on equal hardware.

3.2 Iso-loss Curves

Figures 2 and 3 show the iso-loss curves for both runs. Each curve plots the best (running-minimum) loss achieved up to each resource level, so for any target loss value the horizontal gap between the two curves gives the resource saving.

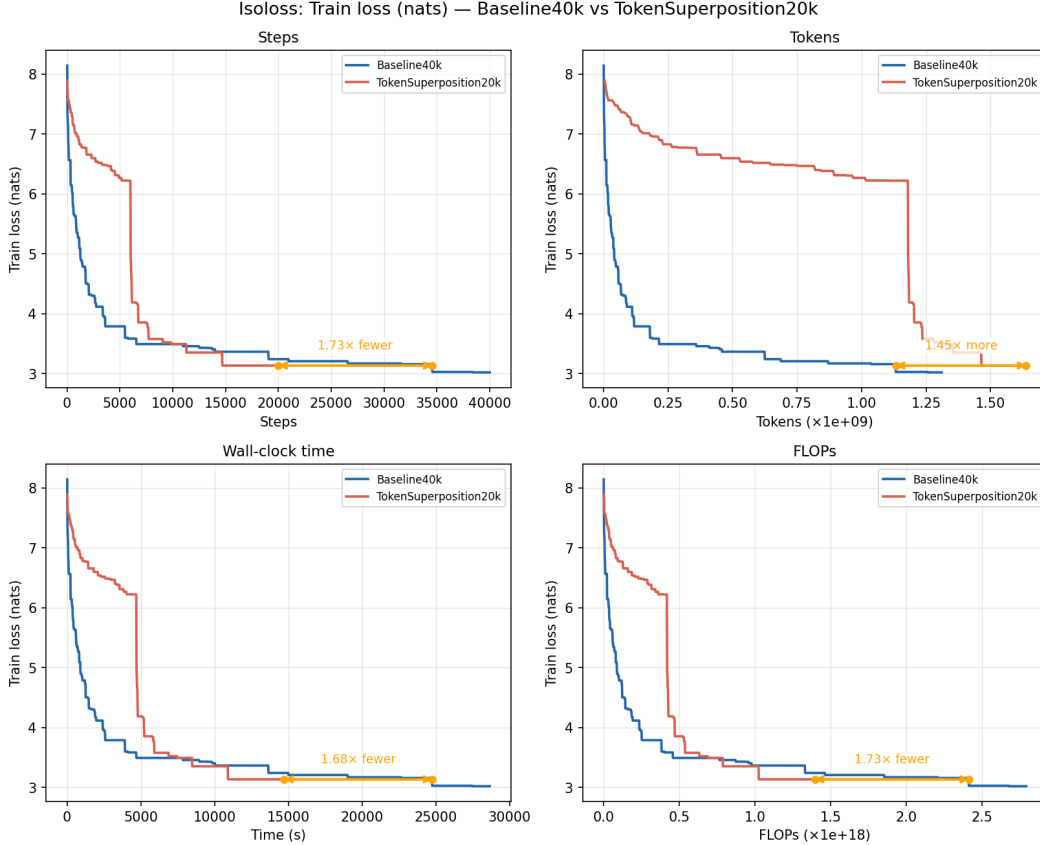


Figure 2: Iso-loss curves for **training loss**. X-axes: gradient steps, data tokens processed, wall-clock seconds, and total FLOPs. Y-axis: running minimum training loss (nats). The red curve (TokenSuperposition20k) initially trains more slowly due to the superposition phase, then recovers sharply and matches the baseline’s training loss efficiency in the final standard-AR phase.

The step-axis panel of Figure 3 clearly shows the characteristic TST profile: the validation loss is initially worse than the baseline (during the superposition phase, the model is optimising a different objective), before a fast recovery once standard training resumes. By the end of the TST run’s 20,000 steps, the baseline requires a further $\sim 18,000$ steps — nearly doubling its compute budget — to reach the same validation loss.

4 Discussion

4.1 Comparison with the Original Paper

The original paper evaluates TST with $s = 6$, $r = 0.3$ on a 270 M parameter model (roughly comparable to our 345 M model in parameter count) trained on the DCLM dataset, reporting a

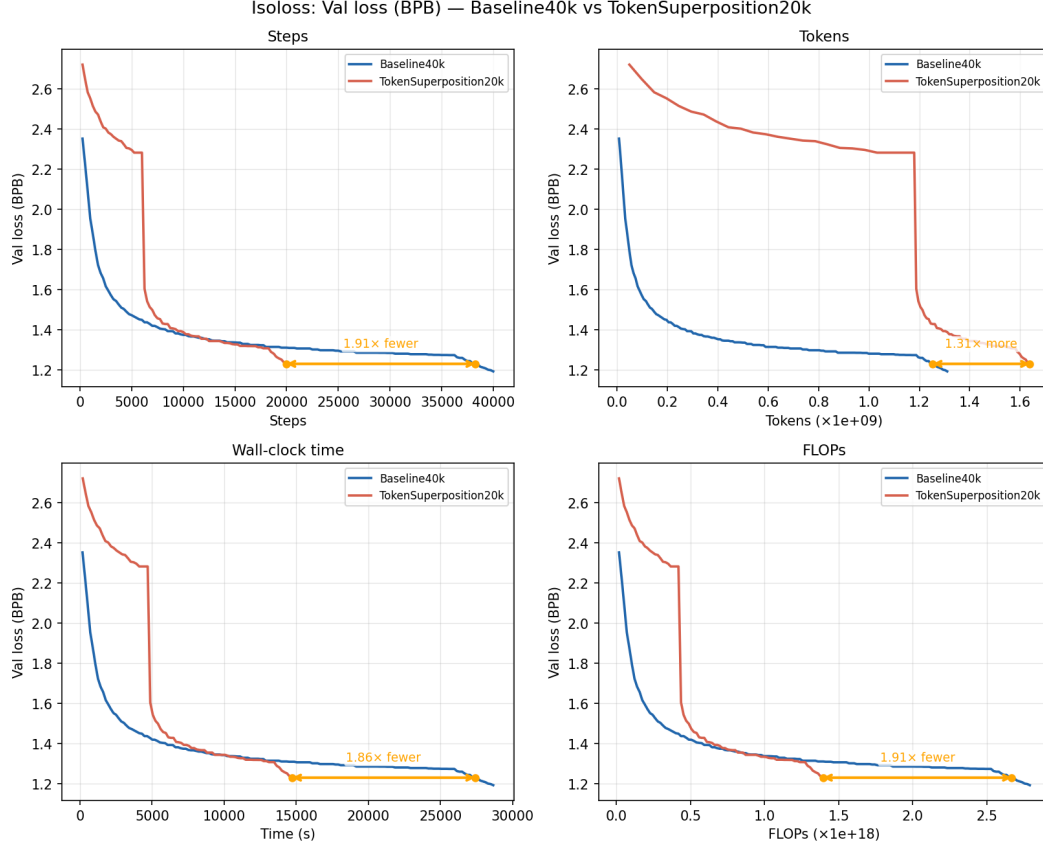


Figure 3: Isoloss curves for **validation cross-entropy**. Same axes as Figure 2. The baseline (blue) is consistently more efficient per step for the first $\sim 20,000$ steps, but the TST run’s recovery phase produces a rapid improvement, and the TST run’s final validation CE of 3.41 nats requires $1.91\times$ more baseline steps to match.

final loss of 3.142 nats for TST versus 3.212 nats for an equal-FLOPs baseline. Our 345 M run on FineWeb-Edu does not achieve the same absolute loss values, but we do not expect an exact match: the dataset (FineWeb-Edu vs. DCLM), model parametrization (untied vs. tied weights in the original, different training duration), and evaluation protocol (validation split from the same dataset vs. held-out data) all differ.

Our reproduction confirms the qualitative finding: at $s = 6$, $r = 0.3$, a TST run achieves a target validation loss approximately twice as fast (in steps and FLOPs) as a standard baseline, on a different model and dataset from those used in the original paper. The $1.91\times$ speedup we observe is broadly consistent with the ratios reported in Peng et al. [6] for the 270 M–600 M scale.

4.2 Implementation Fidelity

The key algorithmic choices that are known to matter for TST and that we correctly reproduced are:

1. *Shared embeddings and LM head.* The original paper shows (via a randomisation ablation) that re-initialising the input embedding and output projection between phases destroys TST’s benefits. Our implementation retains weight tying across both phases.
2. *Equal-FLOPs steps.* We scale the data sequence length by s during the superposition phase so that each gradient step processes the same number of latent positions as the baseline. This ensures that our step-axis and FLOP-axis comparisons are valid.

3. *MCE loss with shared log-partition.* We compute $\log \sum_i \exp(z_i)$ once per latent position and accumulate the s cross-entropy terms, matching the efficiency described in the original paper.

4.3 Limitations

Our reproduction differs from the original paper in several ways:

- **Single run.** We did not perform multiple seeds or a hyperparameter sweep; statistical significance cannot be assessed from a single run.
- **Scale.** We reproduce one setting at one model size (345 M). The original paper validates up to 10 B parameters; we do not assess whether the $1.91\times$ speedup generalises to larger scales.
- **Downstream evaluations.** We report only validation cross-entropy. The original paper also evaluates on ARC, HellaSwag, MMLU, and other benchmarks; we omit these to keep the scope of the reproduction tractable.
- **Dataset.** We use FineWeb-Edu; the original paper uses DCLM for smaller models and a 50/50 DCLM/FineWeb-Edu mix for the 10 B run. Dataset differences can confound absolute loss comparisons.

5 Conclusion

We re-implement Token-Superposition Training from scratch and confirm the central claim of Peng et al. [6]: a TST run with superposition bag size $s = 6$ and step ratio $r = 0.3$ achieves a given validation cross-entropy in approximately half the gradient steps and FLOPs of a standard autoregressive baseline, at the cost of processing more data tokens. Our 345 M parameter reproduction on FineWeb-Edu yields a $1.91\times$ step/FLOP speedup, consistent with the original paper’s results at comparable scale. These results confirm the core claim that TST significantly accelerates pre-training at constant compute cost per step. The method is straightforward to implement — the core changes fit in under 30 lines of PyTorch — and the results transfer to a different model and dataset from those used in the original work.

References

- [1] S. Hu, Y. Tu, X. Han, C. He, G. Cui, X. Long, Z. Zheng, Y. Fang, Y. Huang, W. Zhao, et al. Minicpm: Unveiling the potential of small language models with scalable training strategies. *arXiv preprint arXiv:2404.06395*, 2024.
- [2] A. Karpathy. nanogpt, 2022. URL <https://github.com/karpathy/nanoGPT>.
- [3] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.
- [4] A. Lozhkov, L. Ben Allal, L. von Werra, and T. Wolf. Fineweb-edu: the finest collection of educational content on the web. *Technical Report*, 2024.
- [5] Modal Labs. Modal: Run code in the cloud, 2024. URL <https://modal.com>.
- [6] B. Peng, T. Gigant, and J. Quesnelle. Efficient pre-training with token superposition. *arXiv preprint arXiv:2605.06546*, 2026.
- [7] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners. *OpenAI Blog*, 1(8):9, 2019.